Orden de reuniones con programación dinámica

Explicación paso a paso con ejemplos. Basado en las filminas de Optimización de consultas (Bloque 6) y en el ejercicio del parcial del 06/11/2025.

1. Para qué sirve

La reunión natural es **asociativa**: (S ⋈ T) ⋈ U da el mismo resultado final que S ⋈ (T ⋈ U). Lo que cambia es el tamaño de los **resultados intermedios** que se generan en el camino. Con las tablas S, T y U del parcial, cada una con 1000 tuplas:

- Reunir primero S con T deja un intermedio de 2000 tuplas.
- Reunir primero T con U deja un intermedio de 1000 tuplas.
- Reunir primero S con U deja un intermedio de 1.000.000 de tuplas.

Elegir bien el orden ahorra muchísimo trabajo. Probar todos los órdenes explota: con n tablas hay (2(n−1))! / (n−1)! órdenes (3 tablas: 12, 5 tablas: 1680, 10 tablas: más de 17 mil millones). La programación dinámica evita eso: **resuelve cada subconjunto de tablas una sola vez, anota el resultado en una tabla y lo reutiliza**. Con 4 tablas son 16 subconjuntos en vez de 120 órdenes.

2. Conceptos que usa

a) Tamaño estimado de una reunión (en tuplas). Para r ⋈ s por un atributo común A:

tamaño = tam(r) · tam(s) / max(V(A, r), V(A, s))

Donde V(A, r) es la cantidad de valores distintos de A en r. Si las dos tablas no comparten ningún atributo, la reunión es un **producto cartesiano**: tam(r) · tam(s).

b) Resultado intermedio. Un resultado que el sistema tiene que fabricar en el medio de un plan, porque no existe en la base de datos.

c) Las tablas de la base de datos se asumen cargadas en memoria. Por eso una tabla base no cuesta nada: no hay que fabricarla. Según la filmina, el costo de una tabla sola y el de un par de tablas es 0 porque no hay tablas intermedias involucradas.

d) Partición de un conjunto. Dividir un conjunto S de tablas en dos grupos disjuntos S1 y S2. Cada partición corresponde a un plan: se arma la mejor expresión de S1, la mejor de S2, y se reúnen.

e) Inducción por tamaño. Se llenan primero los subconjuntos de 1 tabla, después los de 2, después los de 3, y así. Cada paso usa lo que ya se calculó.

3. Qué mide el costo

En esta tabla el costo se mide en **cantidad de tuplas**, no en bloques. La regla de la filmina es:

costo de la partición {S1, S2} = mejor costo(S1) + mejor costo(S2) + tamaño(S1) + tamaño(S2)

con dos aclaraciones: el tamaño de una **tabla base no se cuenta** (ya está en memoria), y el tamaño de un **resultado intermedio sí se cuenta** (hubo que fabricarlo).

Lectura práctica (es una interpretación, pero coincide con todos los números de la filmina): el costo de un plan es la **suma de los tamaños de los resultados intermedios que produce**. El resultado final no se cuenta porque es el mismo con cualquier orden y hay que producirlo siempre.

Ojo con la palabra cero: una tabla base tiene tamaño (S tiene 1000 tuplas), pero **ese tamaño no se suma** porque no hay que fabricarla.

Este costo es solo una medida simplificada para decidir el orden. Una vez elegido el plan, el costo real en bloques se calcula aparte, con las fórmulas de los operadores físicos.

4. Cómo es la tabla

Una columna por cada subconjunto de tablas, y tres filas:

Fila	Qué guarda
Size	Cuántas tuplas tiene la reunión de esas tablas
Cost	El menor costo de computarla
Best plan	La expresión que consigue ese costo

5. El algoritmo paso a paso

1. **Conjuntos de 1 tabla.** Size = su tamaño. Cost = 0. Best plan = la tabla.
2. **Conjuntos de 2 tablas (todos los pares).** Size con la fórmula de reunión. Cost = 0. Best plan = la reunión (con la menor a la izquierda). Se calculan **todos** los pares, incluso los que no comparten atributos: ahí el tamaño es el producto cartesiano, sale enorme y ese par pierde, pero hay que anotarlo.
3. **Conjuntos de 3 tablas.** Para cada uno:
   - Se listan todas las particiones posibles (una tabla sola y un par).
   - Se calcula el costo de cada una con la regla de la sección 3.
   - Se elige la de menor costo: es la **partición ganadora**.
   - Cost = ese costo mínimo. Best plan = reunión de los mejores planes de los dos grupos. Size = fórmula de reunión (se calcula una vez, es el mismo con cualquier partición).
4. **Conjuntos de 4 tablas.** Igual, pero hay dos tipos de partición: 2+2 y 3+1.
5. **La respuesta** es la columna que contiene todas las tablas.

La **partición ganadora** no es un par de tablas: es la forma de dividir el conjunto en dos grupos que sale más barata.

6. Ejemplo completo: S, T y U (parcial 06/11/2025, ejercicio 2)

Enunciado. Reunión natural de S(b,c), T(c,d) y U(d,a), cada una con 1000 tuplas. Datos: V(U,a) = 50, V(S,b) = 100, V(S,c) = 500, V(T,c) = 20, V(T,d) = 50, V(U,d) = 1000. Piden: (1) la partición ganadora y el mejor plan, (2) el tamaño de (S ⋈ T) ⋈ U.

Los datos V(U,a) y V(S,b) no se usan, porque a y b no son atributos de reunión.

Paso 1: tablas solas

	{S}	{T}	{U}
Size	1000	1000	1000
Cost	0	0	0
Best plan	S	T	U

Paso 2: los tres pares

	{S,T}	{T,U}	{S,U}
Atributo común	c	d	ninguno
Cuenta	1000·1000 / max(500, 20)	1000·1000 / max(50, 1000)	1000·1000
Size	2000	1000	1.000.000
Cost	0	0	0
Best plan	S ⋈ T	T ⋈ U	S ⋈ U

Paso 3: el conjunto {S,T,U}

Hay tres particiones. En cada una, un grupo es una tabla base: no se fabrica, así que su costo es 0 y su tamaño no se cuenta. El otro grupo es un par: su costo es 0 pero es un resultado intermedio, así que **su tamaño sí se cuenta**.

Partición	Plan	Intermedio que se fabrica	Costo
{{S,T},{U}}	(S ⋈ T) ⋈ U	S ⋈ T, 2000 tuplas	0 + 0 + 2000 = 2000
{{T,U},{S}}	(T ⋈ U) ⋈ S	T ⋈ U, 1000 tuplas	0 + 0 + 1000 = **1000**
{{S,U},{T}}	(S ⋈ U) ⋈ T	S ⋈ U, 1.000.000 de tuplas	0 + 0 + 1.000.000 = 1.000.000

La menor es la segunda. Se anota la columna:

	{S,T,U}
Size	2000
Cost	1000
Best plan	(T ⋈ U) ⋈ S

El Size se calcula una sola vez: |T ⋈ U| = 1000 se reúne con S por c, así que 1000 · 1000 / max(V(T,c), V(S,c)) = 10^6 / max(20, 500) = 2000.

Respuestas

Ítem 1. La partición ganadora es {{T,U},{S}} y el mejor plan es (T ⋈ U) ⋈ S, con costo 1000. Conviene empezar por el par más chico y evitar S ⋈ U, que no comparte atributos y explota.

Ítem 2. El tamaño de (S ⋈ T) ⋈ U: el resultado de S ⋈ T tiene 2000 tuplas y se reúne con U por d, el único atributo común. El d de S ⋈ T viene de T, así que V ≈ V(T,d) = 50. Entonces 2000 · 1000 / max(50, 1000) = 2.000.000 / 1000 = **2000 tuplas**. Coincide con el Size de la columna {S,T,U}: el tamaño final no depende del orden.

7. Ejemplo de las filminas con 4 tablas

El ejercicio del parcial es este ejemplo sin la tabla R. R(a,b), S(b,c), T(c,d), U(d,a), cada una con 1000 tuplas. Datos: V(R,a) = 100, V(R,b) = 200, V(S,b) = 100, V(S,c) = 500, V(T,c) = 20, V(T,d) = 50, V(U,d) = 1000, V(U,a) = 50.

Pares (todos con Cost 0):

Par	Atributo común	Cuenta	Size
{R,S}	b	10^6 / max(200, 100)	5000
{R,T}	ninguno	1000 · 1000	1.000.000
{R,U}	a	10^6 / max(100, 50)	10.000
{S,T}	c	10^6 / max(500, 20)	2000
{S,U}	ninguno	1000 · 1000	1.000.000
{T,U}	d	10^6 / max(50, 1000)	1000

Tríos. Cada trío tiene tres particiones de tipo 2+1, y el costo es el tamaño del par:

Conjunto	Costos de las particiones	Ganadora	Size	Cost
{R,S,T}	{RS}+T = 5000, {RT}+S = 1.000.000, {ST}+R = 2000	(S ⋈ T) ⋈ R	10.000	2000
{R,S,U}	{RS}+U = 5000, {RU}+S = 10.000, {SU}+R = 1.000.000	(R ⋈ S) ⋈ U	50.000	5000

Conjunto	Costos de las particiones	Ganadora	Size	Cost
{R,T,U}	{RT}+U = 1.000.000, {RU}+T = 10.000, {TU}+R = 1000	(T ⋈ U) ⋈ R	10.000	1000
{S,T,U}	{ST}+U = 2000, {TU}+S = 1000, {SU}+T = 1.000.000	(T ⋈ U) ⋈ S	2000	1000

Ejemplo de cálculo de un Size: |(S ⋈ T) ⋈ R| = 2000 · 1000 / max(100, 200) = 10.000, porque el atributo común es b, y el b del par viene de S (V = 100) mientras que el de R es 200.

El conjunto {R,S,T,U}. Hay dos tipos de partición:

- **2+2:** los dos pares son intermedios, así que se suman los **dos tamaños** (los costos de los pares son 0).
- **3+1:** el trío es intermedio, así que se suma **su costo más su tamaño**. La tabla sola no se cuenta.

Partición	Cuenta	Costo
{{R,S},{T,U}}	5000 + 1000	6000
{{R,T},{S,U}}	1.000.000 + 1.000.000	2.000.000
{{R,U},{S,T}}	10.000 + 2000	12.000
{{R,S,T},{U}}	2000 + 10.000	12.000
{{R,S,U},{T}}	5000 + 50.000	55.000
{{R,T,U},{S}}	1000 + 10.000	11.000
{{S,T,U},{R}}	1000 + 2000	**3000**

El plan elegido es ((T ⋈ U) ⋈ S) ⋈ R con costo 3000. Se comprueba contando los intermedios de ese plan: T ⋈ U (1000 tuplas) y (T ⋈ U) ⋈ S (2000 tuplas). El resultado final no se cuenta. Total 1000 + 2000 = 3000.

8. Variante: árboles de reunión profunda a la izquierda

En estos árboles el lado derecho de cada reunión es siempre una tabla base. Se simplifica el algoritmo: no hay particiones 2+2, solo 3+1. Para un conjunto S con k tablas, para cada tabla r de S se calcula costo(S − {r}) + tamaño(S − {r}), y se toma el r que da el menor valor.

La complejidad baja a $O(n \cdot 2^n)$.

9. Errores típicos

1. Sumar el tamaño de una tabla base (no se cuenta, ya está en memoria).
2. Creer que costo 0 significa tamaño 0: son cosas distintas.
3. Usar costos de bloques en esta tabla: acá el costo es en tuplas intermedias.
4. En una partición 2+2, olvidar sumar los dos tamaños. En una 3+1, olvidar sumar el costo del trío además de su tamaño.
5. Saltear los pares que no comparten atributos: hay que anotarlos con el producto cartesiano.
6. Confundir partición ganadora con el par de tablas más chico: es la forma de dividir el conjunto con menor costo.
7. Pensar que el tamaño del resultado final cambia con el orden: el final es el mismo, lo que cambia son los intermedios.

10. Ejercicio para practicar

Tres tablas: A(x,y) con 100 tuplas, B(y,z) con 200 tuplas, C(z,w) con 50 tuplas. Datos: V(A,y) = 20, V(B,y) = 50, V(B,z) = 10, V(C,z) = 10.

Armá la tabla completa y respondé: ¿cuál es la partición ganadora, el mejor plan y el tamaño del resultado de reunir las tres?

Solución

Pares:

- {A,B} comparten y: 100 · 200 / max(20, 50) = 400.
- {B,C} comparten z: 200 · 50 / max(10, 10) = 1000.
- {A,C} no comparten nada: 100 · 50 = 5000.

Trío {A,B,C}:

- {{A,B},{C}} cuesta 400.
- {{B,C},{A}} cuesta 1000.
- {{A,C},{B}} cuesta 5000.

La partición ganadora es **{{A,B},{C}}**, el mejor plan es **(A ⋈ B) ⋈ C** y el costo es **400**. El tamaño del resultado: (A ⋈ B) tiene 400 tuplas y se reúne con C por z, con V(AB,z) = V(B,z) = 10 y V(C,z) = 10, así que 400 · 50 / 10 = **2000 tuplas**. Comprobación con otro orden: (B ⋈ C) ⋈ A por y da 1000 · 100 / max(50, 20) = 2000, el mismo resultado.